Machine Learning for Network Security & Management

1st Rafay Syed
Texas A&M University
College Station, Texas
rafaysyed@tamu.edu

2nd Nicholas Martinez
Texas A&M University
College Station, Texas
nickmartinez@tamu.edu

3rd Carmichael Rivera
Texas A&M University
College Station, Texas
carmichael0310@tamu.edu

***Abstract*—This project examines how machine learning can be used for DDoS attack detection and encrypted traffic classification. Using flow-based datasets, the models were trained to classify traffic as Normal or Attack in Track A and as Browsing or Streaming in Track B. Decision Tree, Random Forest, and XGBoost were evaluated, with a Dummy Classifier used as a baseline. The results showed that all trained models clearly outperformed the baseline, with XGBoost giving the best overall balance of accuracy, F1-score, and efficiency, while Decision Tree offered the lowest inference cost for a trained model on a resource constrained deployment. Overall, the project shows that machine learning can effectively support intrusion detection and encrypted traffic analysis when traditional methods are limited.**

Keywords—machine learning, network security, DDoS detection, intrusion detection system

I. Introduction

Modern network security is currently struggling with two different crises, automated network attacks on top of end-to-end encryption becoming more and more commonplace. Traditional or normal security systems are struggling to maintain efficiency at the larger scales that are developing, and are being faced with a lot of threats that they're struggling to handle without hurting their user's right to privacy.

Traditionally, networks use defenses like firewalls and Access Control Lists(ACLs), which operate based on known signatures and very unchanging sets of rules, blocking specific ports or certain ranges of IP addresses. Modern Distributed Denial of Service (DDoS) threats are often executed by using IP spoofing however and can bypass this sort of gatekeeping. Machine Learning (ML) allows our IDS to move past the static ruleset and analyze the flow and shape of the traffic to try and determine its danger level. By evaluating these factors (inter-arrival times, header distributions, etc.) ML can identify the behavioral signatures of a DDoS flood, without being dependent on the IP.

Considering how common HTTPS encryption has become, Deep Packet Inspection (DPI) has become largely useless for traffic management [3]. These encryption algorithms hide packet payloads, which is an issue for DPI, but ML addresses this by classifying the data based on the metadata surrounding the packet. Analyzing things like 'flowBytesPerSecond' allows this model to accurately categorize traffic. It can distinguish between streaming and intermittent browsing and doesn't even need to utilize any of the encrypted data. This project evaluates the effectiveness of three main ML classifiers given, 'Decision Tree', 'Random Forest', and 'XGBoost'. Our group will be doing both tracks A & B, where we cover the detection of Distributed Denial of Service (DDoS), and encryption traffic classification (e.g. Streaming vs Browsing).

The implications in the real world of this ML technology range well beyond network security. For example this ability to differentiate traffic can be used for service providers to be able to identify what users are taking up the most bandwidth through analysis of their network traffic could allow them to reallocate accordingly. This could seriously mitigate severe network congestion during peak usage times and create an overall more positive network experience.

II. Data Preparation

A. Track A dataset

The CICDDoS2019 dataset [1] was created for DDoS-related network traffic research and is meant to support intrusion detection and traffic classification studies using flow-based features rather than raw packet contents. In the project notebook, the final Track A file contained 30 traffic-related features after cleaning, including timing features, packet length statistics, packet/byte rate features, direction indicators, and forward-flow measurements. These features matter because DDoS traffic often behaves differently from normal traffic. Flow duration can reveal unusually short or repeated attack flows, packet counts can reflect traffic flooding, and byte counts or bytes-per-second can show high-volume behavior. Together, these features and data help the model detect the behavioral signature of an attack rather than just its exact content.

B. Track B dataset

The Track B dataset comes from the ISCX-VPN-NonVPN-2016 traffic-classification [2] setting, where the focus is understanding encrypted traffic using flow-level statistics instead of payload inspection. In the notebook, the dataset was reduced to a binary problem with Browsing and Streaming as the two classes. Timing and rate features matter in this kind of dataset because encrypted traffic still has observable behavior even when the content is hidden. Streaming traffic usually lasts longer and moves data more continuously, while browsing traffic is usually more bursty and stop-and-start. That is why features like flow duration, bytes per second, packets per second, and inter-arrival times are

strong indicators of application behavior even without direct content inspection.

C. Separate Training and Testing data

Separating data into training and test sets is important because it gives a more honest evaluation of the models. The training set is where the model learns the patterns in the traffic, while the test set is used to check how well the model performs on unseen data. If the same data were used for both training and testing, the results could be misleading because the model might simply memorize the training data rather than learning general traffic behavior. In this project, Track A already used separate train and test files, while Track B used an 80/20 stratified split so that the class distribution stayed consistent across both sets. This separation is necessary to show whether the model can generalize to new network traffic instead of only performing well on data it has already seen.

D. Binary Labels for Attack Vs Normal

The original Track A labels contained multiple attack classes, but in the notebook transformed them into a binary problem where normal traffic became 0 and all attack traffic became 1. This simplifies the project because it reduces the number of decisions the model has to make. Instead of learning how to separate many specific DDoS variants from each other, the model only needs to decide whether a flow is normal or malicious. That makes the classification problem easier to train and interpret. This dataset is more aligned with the role of a first stage IDS, where the main goal is to detect suspicious traffic as quickly as possible.

E. Class Imbalance in Track B

Class imbalance means that one class appears much more often than the other. In Track A, the training labels were about 90.85% attack and 9.15% normal, and the test labels had almost the exact same proportions. In Track B, the training labels were about 83.92% browsing and 16.08% streaming, and the test labels were also very close to that split. Because of this imbalance, raw accuracy can be misleading. A model can predict the majority class most of the time and still get a high accuracy score without actually handling the minority class well. Precision, Recall, and F1-score are more meaningful because they show how well the model handles positive detections, missed cases, and the balance between those two. This is especially important in cybersecurity, where missing the minority class can still be a serious failure even if overall accuracy looks good.

F. Missing or Infinite Values Handling

The notebook handled missing and infinite values by cleaning the data before training. In Track A, the feature columns were converted to numeric values using *pd.to_numeric(errors='coerce')*, then any inf or -inf values were replaced with NaN, and the missing values were filled with zero. In Track B, the dataset was cleaned by replacing inf and -inf with NaN and then dropping rows with missing values. These steps were necessary because machine learning models require consistent numerical input, and leaving invalid values in the dataset could either crash the training process or lead to unreliable predictions.

III. ALGORITHMS

A. Models

1. Dummy Classifier

The Dummy Classifier model does not learn any new data but serves to be used as a baseline for comparison. By using a stratified strategy it can make predictions by randomly assigning labels based on the same class distribution as the dataset. This means that it predicts browsing and streaming roughly the same proportions as can be seen in test data. The model is useful in showing the minimum level of performance expected with a very surface level depth to it. The parameters used were *strategy='stratified'* to account for class imbalance and *random_state=42* for reproducibility. Since there was not much training the training was done fast.

2. Decision Tree

The Decision Tree model works by splitting the dataset into smaller groups based on their values that separate both normal and attack traffic [5]. At each step the model then selects a feature and a threshold that maximizes the separation between classes. This continues until a final decision has been made which forms a tree like structure. This model uses the default parameters since the dataset is already structured and the goal is to get an idea without overcomplicating it. This Decision Tree trained quickly within a second of making it. For Track B the parameters used were *max_depth=10* to limit how deep a tree can grow and *random_state=42* in order to ensure consistent results.

3. Random Forest

The Random Forest model works by creating multiple decision trees instead of one [4]. Each tree is trained on a random group of data and features and the final output is made by combining the outputs of all trees. The parameter *n_estimators=100* was used, just so that the model builds 100 trees. I thought this value was good for accuracy and computation time. Also a random_state was used to make sure there was consistency between all the runs. Random Forest generally has a higher training cost than a single Decision Tree because it constructs an ensemble of multiple trees rather than relying on one tree. For Track B *max_depth=10* was also used and *random_state=42* in order to control depth and ensure reproducibility.

4. XGBoost

The XGBoost model works by building decision trees sequentially meaning that a new tree works on fixing the errors that were made by previous ones [6]. This approach allows each iteration to capture patterns and also provide a result which is

higher in accuracy. The model was initialized with *eval_metric = 'logloss'* which is appropriate for binary problems and also the *use_label_encoder =False* to avoid any warnings since the labels were already provided as 0 and 1. XGBoost also tends to have a higher training cost because it uses a boosting process, where trees are built one after another to improve earlier predictions. However it only took a couple of seconds to train and was much quicker than the random forest. For track B n_estimators-100 was used for the number of boosting rounds and a max depth=6 to prevent overfitting.

B. Random Forest Ensemble and Bagging

Random Forests improves on a single decision tree by using an ensemble method named bagging. Instead of only relying on one tree it can build many different decision trees using random subsets of the training data and features. Each tree will have its own prediction and the final result is only determined by viewing the bigger picture of all the trees. This approach is effective with reducing overfitting which can be a common problem with a single Decision tree. A single tree can be too niche and only work on the training data vs new data. It's by a collection of trees that Random Forest is able to average out errors and provide a more accurate prediction. So it can better handle unseen network traffic than a single Decision Tree.

C. XGBoost VS. Random Forest

XGBoost is different from Random Forest in how the trees are built and then combined. Random Forest uses bagging when there are multiple trees which are trained independently on random subsets of data and the predictions are determined through voting. Each tree works separately from the other with the goal to reduce variance by averaging many models. XGBoost uses boosting, which is when the trees are built sequentially rather than independently. Each new tree focuses on correcting the errors made by the previous tree. This makes sure that the model is able to learn from its mistakes and improve. It's due to this difference that Random Forest will be faster but XGBoost is more powerful since it can achieve a greater accuracy score by being able to capture more complex patterns. Yet its drawback is that it takes longer to train.

IV. Results & Model Selection

A. Accuracy, Precision, Recall, and F1-Score

Accuracy measures the fraction of total predictions that were correct, so its formula is $(TP+TN)/(TP+TN+FP+FN)$. Precision measures how many predicted positive cases were actually positive, so its formula is $TP/(TP+FP)$. Recall measures how many actual positive cases were correctly found, so its formula is $TP/(TP+FN)$. F1-score is the harmonic mean of Precision and Recall, so its formula is $2\times(Precision\times Recall)/(Precision+Recall)$. These metrics matter because each one describes a different part of model behavior, and together they give a fuller picture than any single metric alone.

B. Confusion Matrix

A confusion matrix shows how many predictions fell into four groups: true positives, true negatives, false positives, and false negatives. In Track A, a true positive means attack traffic correctly predicted as attack, while a true negative means normal traffic correctly predicted as normal. A false positive means normal traffic was incorrectly flagged as attack, and a false negative means attack traffic was incorrectly missed and labeled as normal. In Track B, a true positive means streaming traffic correctly identified as streaming, while a true negative means browsing traffic correctly identified as browsing. A false positive means browsing traffic was incorrectly labeled as streaming, and a false negative means streaming traffic was missed and labeled as browsing. This is useful because it shows not just how often the model is right, but also what kind of logic errors it is making.

C. Use of Multiple Metrics

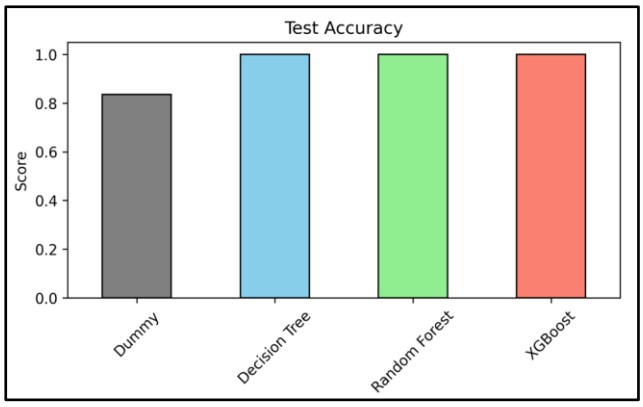
It is important to use multiple metrics because both datasets in this project are imbalanced, which means accuracy alone can make weak models look better than they really are. In Track A, even the Dummy baseline achieved 0.8356 accuracy simply because attack traffic was the majority class. In Track B, the Dummy baseline reached 0.7164 accuracy, but it still did a poor job detecting streaming traffic, which is the minority class. That is why Precision, Recall, F1-score, and confusion matrices matter so much here. They show whether the model is actually identifying the smaller and more important class well, instead of just guessing the majority class often enough to inflate its accuracy.

D. Track A Model Comparations

TABLE I. Track A Model Performance

Model	**Evaluation Metrics**			
	Accuracy	***Precision***	***Recall***	***F1-Score***
Dummy	0.8356	0.91	0.91	0.91
Decision Tree	0.9997	~1.00	~1.00	~1.00
Random Forest	0.9997	~1.00	~1.00	~1.00

Model	Evaluation Metrics			
	Accuracy	*Precision*	*Recall*	*F1-Score*
XGBoost	0.9997	~1.00	~1.00	~1.00



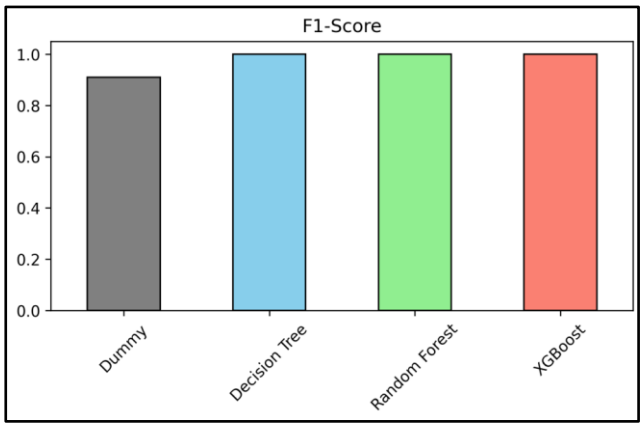
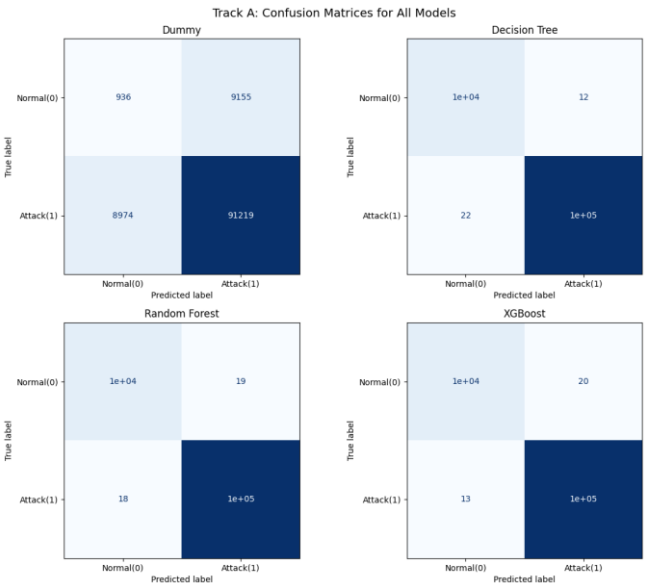


Fig. 1. Test Accuracy Track A

Fig. 2. F1 Score Track A

Fig. 3. Confusion Matrices Track A

In Track A, the learned models performed dramatically better than the Dummy baseline. The Dummy model reached 0.8356 accuracy, but its confusion matrix showed many mistakes because it was



still heavily influenced by the class imbalance. By contrast, the Decision Tree, Random Forest, and XGBoost all reached about 0.9997 accuracy and had near-perfect Precision, Recall, and F1-score for the positive attack class. Their confusion matrices also showed only a very small number of mistakes compared with the full 110,284-row test set. Among these three, Random Forest and XGBoost were both extremely strong, but XGBoost is the better overall choice because it matched the near-perfect performance while using far less training and inference time than Random Forest.

E. Track B Model Comparations

TABLE II. TRACK B MODEL PERFORMANCE

Model	Evaluation Metrics			
	Accuracy	*Precision*	*Recall*	*F1-Score*
Dummy	0.7164	0.13	0.14	0.14
Decision Tree	0.9715	0.91	0.91	0.91
Random Forest	0.9832	0.97	0.93	0.95
XGBoost	0.9849	0.97	0.94	0.95

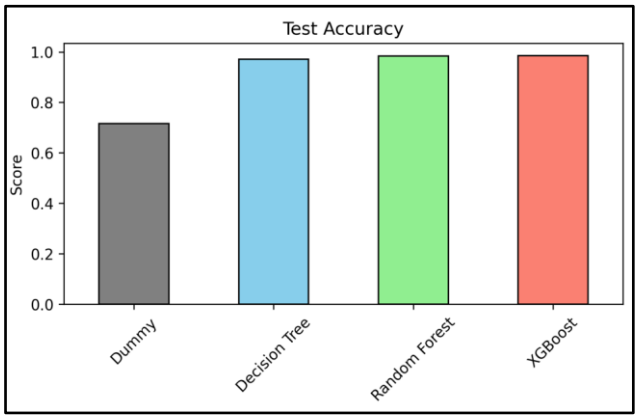


Fig. 4. Test Accuracy Track B

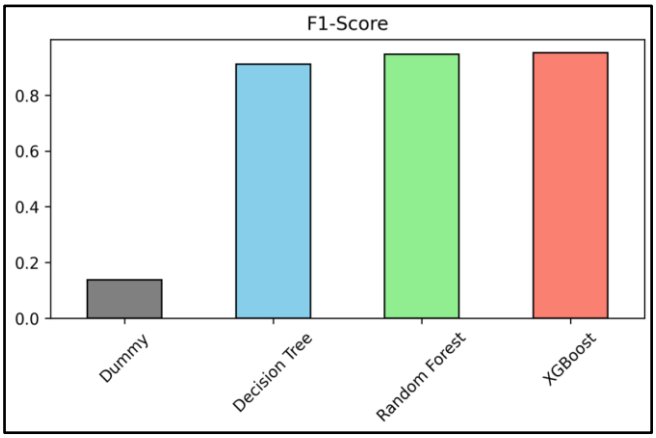


Fig. 5. F1 Score Track B

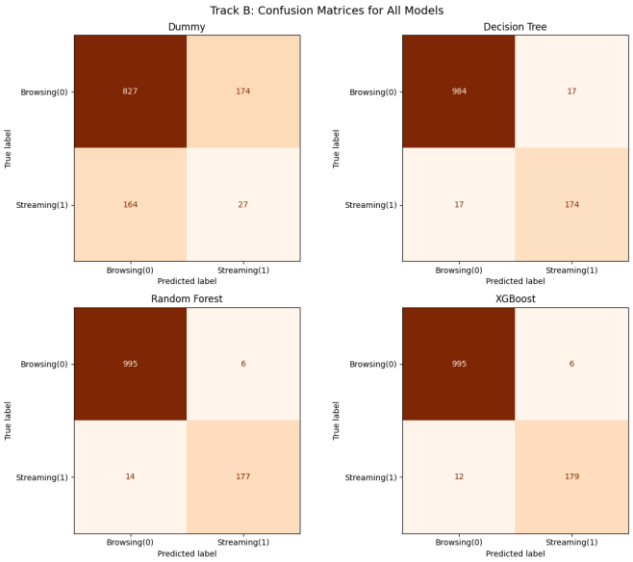


Fig. 6. Confusion Matrices Track B

In Track B, the models showed a more gradual and useful comparison. The Dummy baseline had 0.7164 accuracy and a very weak F1-score for the minority streaming class, which proved that the baseline was not good enough. The Decision Tree improved performance sharply to 0.9715 accuracy with an F1-score around 0.91. Random Forest improved again to 0.9832 accuracy with an F1-score around 0.95, and XGBoost performed slightly better still at 0.9849 accuracy with an F1-score around 0.95 as well. The confusion matrices show that XGBoost made the fewest misses on streaming traffic, which is important because streaming is the minority class. For that reason, XGBoost performed best overall in Track B because it gave the strongest balance of accuracy, F1-score, and operational efficiency.

F. Feature Importance in Track B

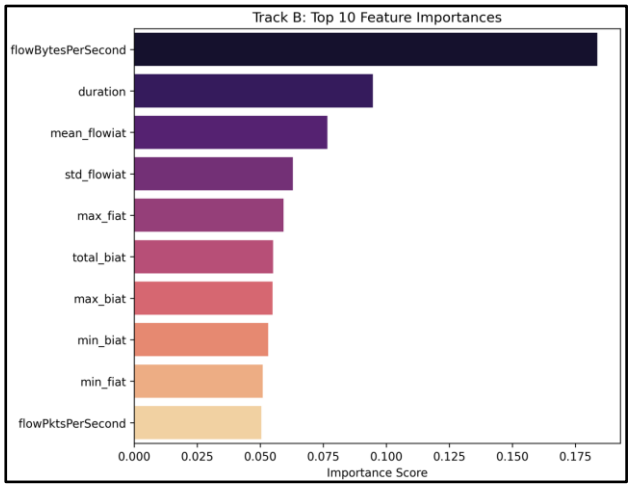


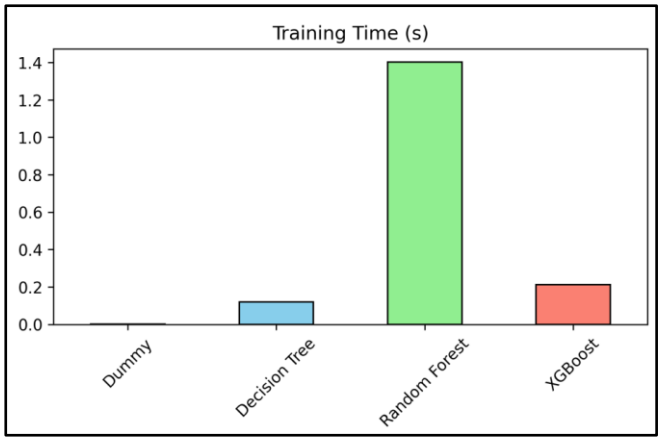
Fig. 7. Top 10 Feature Importance Track B

The Track B feature-importance plot shows that the Random Forest relied most on flowBytesPerSecond followed by other time based features. This reveals that the difference between streaming and browsing traffic is mainly behavioral time of traffic. Streaming traffic usually lasts longer, carries more continuous data, and has more regular flow timing, while browsing traffic is more bursty and session-based. Because of that, byte rate, duration, and timing features become the strongest signals for telling the two classes apart. The feature-importance results make sense from a networking perspective, which supports the idea that the model is learning meaningful traffic behavior instead of random patterns.

G. Training and Inference times

TABLE III. TRACK A AND B MODEL PERFORMANCE

Track	**Model**	**Evaluation Metrics**	
		Train Time (s)	***Inference Time (s)***
A	Dummy	0.0364	0.0178
B	Dummy	0.0014	0.0011
A	Decision Tree	10.5729	0.0128
B	Decision Tree	0.1261	0.0017
A	Random Forest	119.0143	0.5165
B	Random Forest	1.6110	0.0196
A	XGBoost	8.3233	0.2260
B	XGBoost	0.2804	0.0063



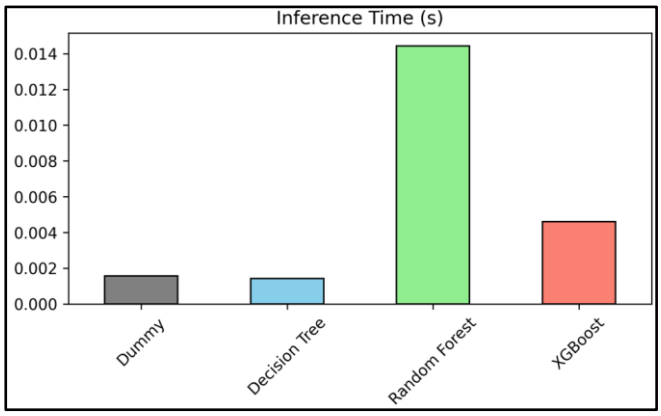


Fig. 8. Track A Training Comparisons

Fig. 9. Track A Inference Comparisons

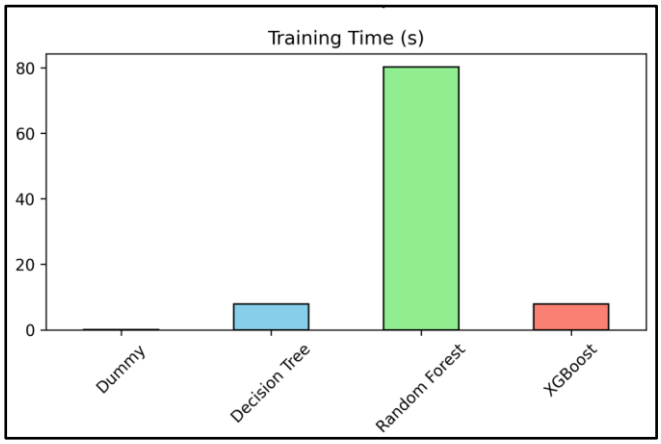


Fig. 10. Track B Training Comparisons

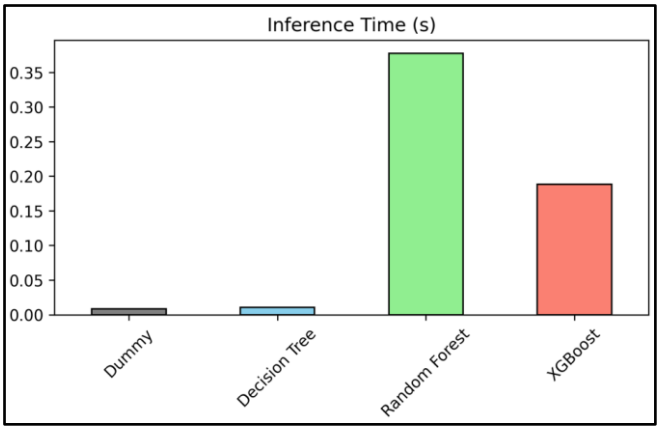


Fig. 11. Track B Inference Comparisons

The training and inference times show that high accuracy is not the only factor that matters. In Track A, Random Forest had the highest overhead by far, training in about 119.01 seconds and taking about 0.5165 seconds for inference, while XGBoost trained in about 8.32 seconds and inferred in about 0.2260 seconds. The Decision Tree was also much lighter than Random Forest, with about 10.57 seconds of training and only about 0.0128 seconds of inference. In Track B, the same pattern appeared again, where Random Forest was the heaviest strong model, while XGBoost gave very strong results at much lower cost. In terms of model size and memory footprint, Decision Tree is the lightest option because it stores only a single tree, Random Forest is the heaviest because it must store many trees at once, and XGBoost offers a better balance by achieving strong performance without the same memory overhead as a large Random Forest. For a resource-constrained device such as an edge router or IoT security system, the best lightweight option is Decision Tree because of its extremely low inference cost. However, if the device can handle a bit more computation, XGBoost is the best overall deployment choice because it keeps very high accuracy and F1 performance while remaining much more efficient than Random Forest.

V. Further Exploration

Deploying AI-powered intrusion detection systems and traffic classifiers in real networks raises important ethical and societal concerns beyond technical performance. One major issue is privacy, because even if encrypted payloads are not directly inspected, flow-based models can still infer sensitive patterns about user behavior, such as whether someone is streaming, browsing, gaming, or using various online services. Another concern is bias and generalization, since a model trained on one dataset may perform well in a controlled environment but behave unfairly or inaccurately on different networks with different users, devices, and traffic patterns. This can lead to false positives, missed attacks, or unequal treatment of certain applications or user groups. There is also the problem of attack surface expanding to the AI IDS. Once machine learning becomes part of the defense system, attackers

may try to poison or manipulate the model itself, turning the security tool into another target. Because of these risks, AI-powered IDS and traffic classifiers should be used as carefully monitored support systems rather than unquestioned replacements for human judgment and traditional network defenses.

VI. APPENDIX I

TRACK A AND B NOTEBOOK:

https://colab.research.google.com/drive/1lCADmNq-eOjPeYANZ1Vbpmh-gvmbtNls?usp=sharing

REFERENCES

[1] Canadian Institute for Cybersecurity, "DDoS evaluation dataset (CIC-DDoS2019)," University of New Brunswick, Fredericton, NB, Canada, 2019.

[2] Canadian Institute for Cybersecurity, "VPN-nonVPN dataset (ISCXVPN2016)," University of New Brunswick, Fredericton, NB, Canada, 2016.

[3] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in Proc. 2nd Int. Conf. Information Systems Security and Privacy (ICISSP), Rome, Italy, 2016, pp. 407–414, doi: 10.5220/0005740704070414.

[4] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[5] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, Classification and Regression Trees. Belmont, CA, USA: Wadsworth International Group, 1984.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16), San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.